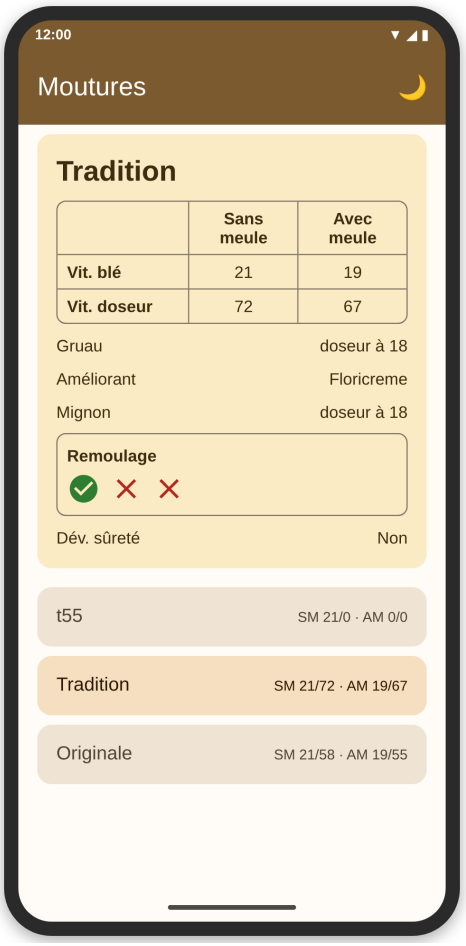
L'application Moutures

Cours pour débutants — Kotlin et Jetpack Compose
Version sans JSON : les données en classes Kotlin



Thème clair



Thème sombre

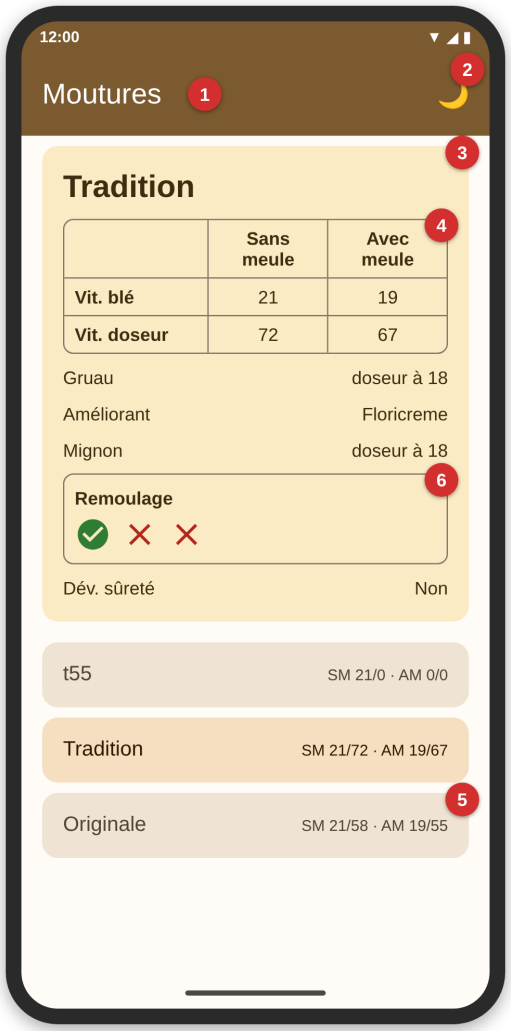
Dans ce cours, vous construisez pas à pas une application Android qui affiche des moutures et leurs réglages, avec un **thème clair et un thème sombre**. Les données sont écrites directement en **Kotlin**, sans fichier JSON. Tous les fichiers du projet sont fournis dans le zip.

Sommaire

1. Ce que fait l'application 3
2. Ouvrir et lancer le projet 5
3. Les data class 7
4. Les données : MouturesData 9
5. Le ViewModel et l'état 11
6. Les bases de Jetpack Compose 13
7. L'écran principal 14
8. Les composants 17
9. Le thème clair et le thème sombre 22
10. Les aperçus @Preview 27
11. Les bibliothèques 28
12. Erreurs fréquentes et solutions 30
13. Exercices 31
14. Glossaire 32

1. Ce que fait l'application

L'application affiche une liste de **moutures** (t55, Tradition, Originale) et, en haut de l'écran, une **carte de détail** avec les réglages de la mouture choisie. Toucher une autre ligne change la carte.



1. La **barre du haut** avec le titre.
2. Le **bouton de thème** : lune = passer en sombre, soleil = repasser en clair.
3. La **carte de détail** de la mouture sélectionnée.
4. Le **tableau** des vitesses, sans meule et avec meule.
5. La **liste** des moutures ; la ligne choisie est colorée.
6. Le **remoulage** : coche verte = oui, croix rouge = non.

Figure 1 — Les parties de l'écran (maquette fidèle aux couleurs et dimensions du code).

Au lancement, la **première mouture est déjà sélectionnée** : la carte est visible tout de suite.

Pourquoi des classes plutôt qu'un JSON ?

Dans la première version, les données étaient dans un fichier **rec.json** lu au démarrage. Ici, on les écrit directement en Kotlin. Comme le JSON était embarqué dans l'application, il fallait de toute façon recompiler pour changer une valeur : on ne perd rien, et on gagne beaucoup.

	JSON dans les assets	**Classes Kotlin (ce cours)**
Faute de frappe	Découverte au lancement (plantage ou valeur à 0)	Signalée **dans l'éditeur**, avant même de lancer
Code nécessaire	Lecture du fichier, conversion, coroutine	**Aucun** : la liste est prête
États de l'écran	chargement, erreur, données	seulement les données

	JSON dans les assets	Classes Kotlin (ce cours)
Modifier une valeur	Recompiler	Recompiler

Le même contenu, deux écritures

```
// Avant (rec.json)
"Tradition": {
  "sm": { "vb": 21, "vd": 72 },
  "am": { "vb": 19, "vd": 67 },
  ...
}
```

```
// Maintenant (MouturesData.kt)
Mouture(
    nom = "Tradition",
    sansMeule = Dosage(vitesseBle = 21, vitesseDoseur = 72),
    avecMeule = Dosage(vitesseBle = 19, vitesseDoseur = 67),
    ...
)
```

Bon à savoir
On en profite pour donner des **noms clairs** : `sm` devient `sansMeule`, `vb` devient `vitesseBle`. Avec le JSON, les noms devaient suivre le fichier.

2. Ouvrir et lancer le projet

Le zip contient le dossier **Moutures** (le projet) et ce cours.

- **Décompressez** le zip.
- Dans **Android Studio** : File → Open, choisissez le dossier **Moutures**.
- Attendez la fin de la **synchronisation Gradle** (barre en bas). La première fois, Android Studio télécharge les bibliothèques : il faut Internet.
- Branchez un téléphone (mode développeur activé) ou créez un émulateur, puis cliquez sur **Run ▶**.

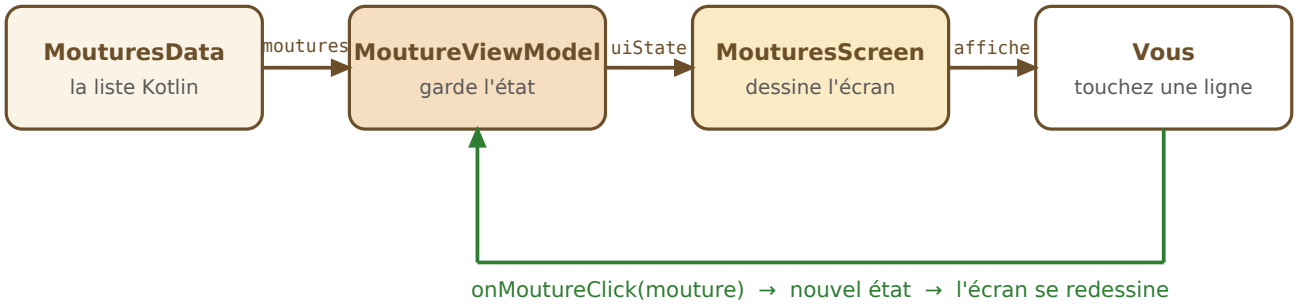
> **Bon à savoir**
> Avec un éditeur sur téléphone comme **CodeAssist**, ouvrez le dossier Moutures de la même façon. Si votre éditeur range le code dans `src/main/kotlin` au lieu de `src/main/java`, déplacez simplement le dossier `com` : les deux fonctionnent.

Le contenu du projet

```
Moutures/
├── settings.gradle.kts           ← nom du projet, dépôts de bibliothèques
├── build.gradle.kts              ← plugins (Android, Kotlin, Compose)
├── gradlew, gradle/              ← lanceur Gradle
└── app/
    ├── build.gradle.kts          ← réglages de l'application + bibliothèques
    └── src/main/
        ├── AndroidManifest.xml   ← déclare l'application et son écran
        ├── res/values/           ← nom de l'appli, thème de démarrage clair
        ├── res/values-night/     ← thème de démarrage sombre
        └── java/com/example/moutures2/
            ├── MainActivity.kt          ← point d'entrée, choix clair/sombre
            ├── data/
            │   ├── Mouture.kt           ← les data class
            │   └── MouturesData.kt      ← la liste des moutures
            └── ui/
                ├── MoutureViewModel.kt  ← l'état de l'écran
                ├── MouturesScreen.kt    ← l'écran principal
                ├── Composants.kt        ← carte, tableau, icônes…
                └── theme/
                    ├── Color.kt         ← les couleurs claires et sombres
                    └── Theme.kt         ← le thème Material 3
```

Comment les pièces s'emboîtent

Les données descendent…



…les événements remontent.

Figure 2 — Les données descendent vers l'écran, les événements remontent vers le ViewModel.

À retenir
Trois dossiers, trois rôles : **data** = les informations, **ui** = ce qu'on voit, **ui/theme** = les couleurs.
Ranger le code ainsi le rend facile à retrouver.

3. Les data class

Une **data class** est une classe faite pour **contenir des valeurs**. On en écrit trois, emboîtées comme des poupées russes :



Figure 3 — Une Mouture contient deux Dosage et une InfoMouture.

Mouture.kt

```
package com.example.moutures2.data

/**
 * Les réglages de vitesse pour un mode de mouture (sans meule ou avec meule).
 */
data class Dosage(
    val vitesseBle: Int,
    val vitesseDoseur: Int,
)

/**
 * Les informations complémentaires d'une mouture.
 * Un type suivi de « ? » (String?) peut valoir null : l'information est absente.
 */
data class InfoMouture(
    val gruau: String?,
    val ameliorant: String?,
    val mignon: String?,
    val remoulage: List<Boolean>,
    val devSurete: Boolean,
)

/**
 * Une mouture complète : son nom, ses deux réglages et ses informations.
 */
data class Mouture(
    val nom: String,
    val sansMeule: Dosage,
    val avecMeule: Dosage,
    val info: InfoMouture,
)
```

Les mots à connaître

Écriture	Signification
`val`	Une valeur qui ne change plus une fois donnée.
`Int`, `String`, `Boolean`	Nombre entier, texte, vrai/faux.
`String?`	Le **?** veut dire « peut être **null** », c'est-à-dire absent. La t55 n'a pas de gruau : `gruau = null`.
`List`	Une liste de vrai/faux, comme `listOf(true, false, false)`.
`data class`	Kotlin ajoute tout seul l'égalité (`==`), l'affichage et `copy()`.
`/** … */`	Un commentaire de documentation : il s'affiche quand on survole le nom dans l'éditeur.

4. Les données : MouturesData

Toutes les moutures sont écrites dans une seule liste, dans un `object` :

MouturesData.kt

```
package com.example.moutures2.data

/**
 * Toutes les moutures de l'application, écrites directement en Kotlin.
 *
 * Pour ajouter une mouture : copiez un bloc Mouture(...) complet,
 * collez-le à la fin de la liste (après une virgule) et changez les valeurs.
 */
object MouturesData {

    val moutures: List<Mouture> = listOf(
        Mouture(
            nom = "t55",
            sansMeule = Dosage(vitesseBle = 21, vitesseDoseur = 0),
            avecMeule = Dosage(vitesseBle = 0, vitesseDoseur = 0),
            info = InfoMouture(
                gruau = null,
                ameliorant = null,
                mignon = null,
                remoulage = listOf(false, false, false),
                devSurete = false,
            ),
        ),
        Mouture(
            nom = "Tradition",
            sansMeule = Dosage(vitesseBle = 21, vitesseDoseur = 72),
            avecMeule = Dosage(vitesseBle = 19, vitesseDoseur = 67),
            info = InfoMouture(
                gruau = "doseur à 18",
                ameliorant = "Floricreme",
                mignon = "doseur à 18",
                remoulage = listOf(true, false, false),
                devSurete = false,
            ),
        ),
        Mouture(
            nom = "Originale",
            sansMeule = Dosage(vitesseBle = 21, vitesseDoseur = 58),
            avecMeule = Dosage(vitesseBle = 19, vitesseDoseur = 55),
            info = InfoMouture(
                gruau = "doseur à 38",
                ameliorant = "Mill Tradi",
                mignon = "doseur à 18",
                remoulage = listOf(false, false, false),
                devSurete = false,
            ),
        ),
    )
}
```

Ce qu'il faut comprendre

- `object` crée un objet unique, disponible partout : on écrit `MouturesData.moutures` sans rien créer avant.
- `listOf(a, b, c)` crée une liste qui ne change pas.
- **Les arguments nommés** (`vitesseBle = 21`) rendent la lecture claire : on voit tout de suite à quoi correspond chaque nombre. Ils sont facultatifs, mais très conseillés ici.

- **La virgule finale** après le dernier élément est autorisée en Kotlin : ajouter un élément ne demande pas de toucher la ligne d'avant.

Ajouter une mouture

Copiez un bloc `Mouture(…),` complet, collez-le avant la parenthèse fermante de `listOf` et changez les valeurs. C'est tout : la nouvelle mouture apparaît dans la liste.

> **Piège fréquent**
> Si vous oubliez un champ, Kotlin refuse de compiler : « No value passed for parameter 'devSurete' ». C'est voulu : avec un JSON, l'oubli serait passé inaperçu jusqu'au lancement.

5. Le ViewModel et l'état

L'**état**, c'est ce que l'écran affiche à un instant donné : la liste et la mouture sélectionnée. Le **ViewModel** le garde en mémoire et le fait évoluer.

MoutureViewModel.kt

```
package com.example.moutures2.ui

import androidx.lifecycle.ViewModel
import com.example.moutures2.data.Mouture
import com.example.moutures2.data.MouturesData
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update

/**
 * Tout ce que l'écran doit afficher à un instant donné.
 */
data class MouturesUiState(
    val moutures: List<Mouture> = emptyList(),
    val selection: Mouture? = null,
)

/**
 * Garde l'état de l'écran. Il survit à la rotation du téléphone :
 * la mouture sélectionnée n'est pas perdue.
 */
class MoutureViewModel : ViewModel() {

    private val _uiState = MutableStateFlow(
        MouturesUiState(
            moutures = MouturesData.moutures,
            // Au démarrage, la première mouture est sélectionnée.
            selection = MouturesData.moutures.firstOrNull(),
        )
    )
    val uiState: StateFlow<MouturesUiState> = _uiState.asStateFlow()

    /** Appelé quand l'utilisateur touche une ligne de la liste. */
    fun onMoutureClick(mouture: Mouture) {
        _uiState.update { etat -> etat.copy(selection = mouture) }
    }
}
```

Élément	Rôle
`MutableStateFlow`	Une « boîte » qui contient l'état et prévient quand il change. Elle est privée (`_uiState`) : seul le ViewModel la modifie.
`StateFlow`	La même boîte en lecture seule, donnée à l'écran.
`firstOrNull()`	La première mouture, ou `null` si la liste est vide (pas de plantage).
`update { … copy(…) }`	Remplace l'état par une copie où seule la sélection a changé.



Figure 4 — Le cycle d'un toucher : de l'événement à la recomposition.

À retenir
Le ViewModel **survit à la rotation** de l'écran : tournez le téléphone, la mouture choisie reste sélectionnée. L'écran ne modifie jamais l'état lui-même : il appelle `onMoutureClick`.

Bon à savoir
Comme il n'y a plus de fichier à lire, plus besoin de coroutine, de chargement ni d'erreur : la version JSON du ViewModel faisait le double de lignes.

6. Les bases de Jetpack Compose

Avec Compose, l'interface est décrite par des **fonctions Kotlin** marquées `@Composable`. Quand l'état change, Compose rappelle ces fonctions et met l'écran à jour : la **recomposition**.

```
@Composable
fun Bonjour(nom: String) {
    Text(text = "Bonjour $nom")
}
```

Les trois conteneurs

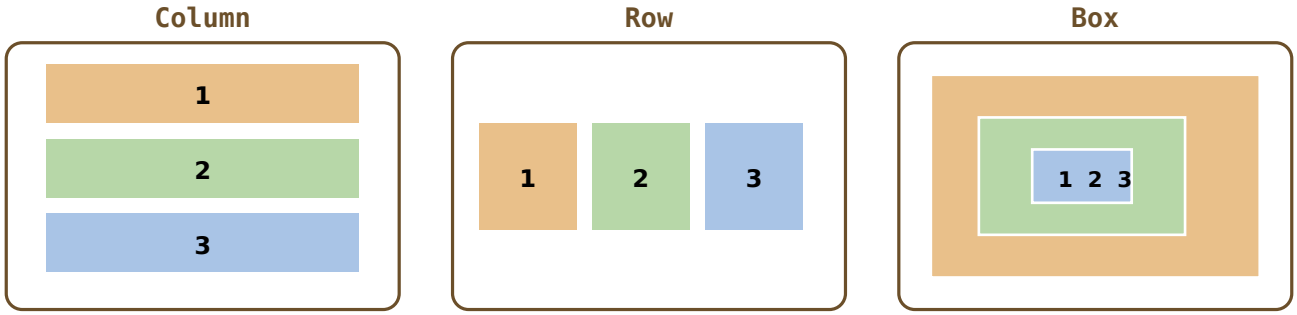


Figure 5 — Column empile vers le bas, Row aligne de gauche à droite, Box superpose.

Le Modifier

Le `Modifier` règle la taille, les marges, le fond, la bordure, le clic. On enchaîne les réglages avec des points, et **l'ordre compte** :

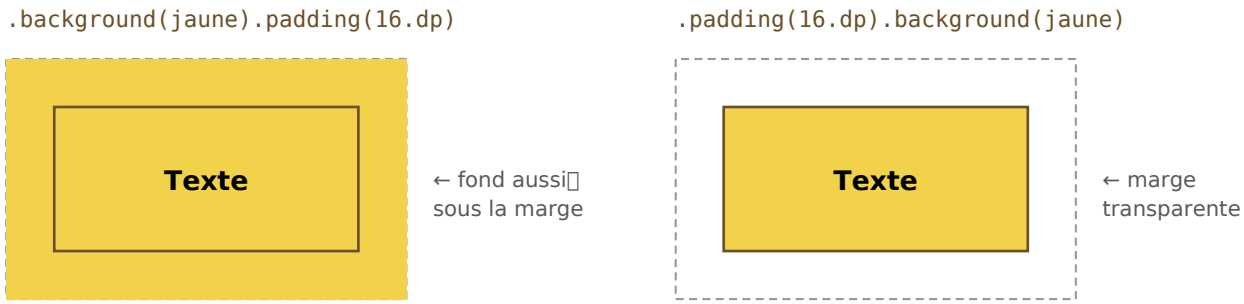


Figure 6 — Le même texte avec deux ordres différents.

Les unités et weight

- **dp** : pour les tailles et marges, même taille physique sur tous les écrans.
- **sp** : pour le texte ; suit en plus la taille de police choisie dans les réglages d'Android.
- `Modifier.weight(1f)` : prend une part de la place restante. N'existe que dans une `Row` ou une `Column`.

Piège fréquent
Une fonction `@Composable` ne peut être appelée que depuis une autre fonction `@Composable`. Sinon : « @Composable invocations can only happen from the context of a @Composable function ».

7. L'écran principal

L'écran est coupé en deux fonctions. `MouturesScreen` récupère le ViewModel ; `MouturesContent` ne reçoit que des données. On peut ainsi afficher `MouturesContent` dans un aperçu sans lancer l'application (chapitre 10).

MouturesScreen.kt — relier l'écran au ViewModel

```
@Composable
fun MouturesScreen(
    sombre: Boolean,
    onSombreChange: (Boolean) -> Unit,
    viewModel: MoutureViewModel = viewModel(),
) {
    val etat by viewModel.uiState.collectAsStateWithLifecycle()
    MouturesContent(
        etat = etat,
        sombre = sombre,
        onSombreChange = onSombreChange,
        onMoutureClick = viewModel::onMoutureClick,
    )
}
```

- `viewModel()` fournit le ViewModel (le même après une rotation).
- `collectAsStateWithLifecycle()` transforme le `StateFlow` en état Compose.
- `by` permet d'écrire `etat` au lieu de `etat.value`.
- `viewModel::onMoutureClick` passe la **fonction** elle-même, sans l'appeler.

MouturesScreen.kt — le contenu

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MouturesContent(
    etat: MouturesUiState,
    sombre: Boolean,
    onSombreChange: (Boolean) -> Unit,
    onMoutureClick: (Mouture) -> Unit,
) {
    val listState = rememberLazyListState()

    // À chaque nouvelle sélection, on remonte en haut pour voir la carte de détail.
    LaunchedEffect(etat.selection) {
        if (etat.selection != null) listState.animateScrollToItem(0)
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Moutures") },
                actions = {
                    // Bouton qui bascule entre le thème clair et le thème sombre.
                    IconButton(onClick = { onSombreChange(!sombre) }) {
                        Text(text = if (sombre) "☀️" else "🌙", fontSize = 20.sp)
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.primary,
                    titleContentColor = MaterialTheme.colorScheme.onPrimary,
                ),
            )
        },
    ) { padding ->
        LazyColumn(
            state = listState,
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp),
        ) {
            // 1) La carte de détail, en tête de liste.
            etat.selection?.let { mouture ->
                item(key = "detail") {
                    DetailMouture(mouture)
                }
            }

            // 2) Une ligne par mouture.
            items(etat.moutures, key = { it.nom }) { mouture ->
                LigneMouture(
                    mouture = mouture,
                    selectionnee = mouture.nom == etat.selection?.nom,
                    onClick = { onMoutureClick(mouture) },
                )
            }
        }
    }
}
```

Les points importants

Élément	Rôle
`Scaffold`	La structure de l'écran : barre du haut + contenu. Son `padding` indique la place prise par les barres ; on l'applique à la liste.
`TopAppBar`	La barre du haut. `actions` place le bouton de thème à droite.
`IconButton`	Un bouton rond. `onSombreChange(!sombre)` inverse le thème (`!` = « le contraire de »).
`LazyColumn`	Une liste qui défile et ne dessine que ce qui est visible.
`item { }` / `items(…) { }`	Dans une `LazyColumn`, on déclare un élément ou un élément par mouture.
`key`	Un identifiant unique par élément, pour que Compose s'y retrouve.
`LaunchedEffect`	Code lancé quand la sélection change : la liste remonte en haut pour montrer la carte.

Piège fréquent
Ne mettez pas une `LazyColumn` dans une `Column` avec `verticalScroll` : l'application plante. C'est pour ça que la carte de détail est un `item` de la liste : en mode paysage, tout défile ensemble.

8. Les composants

Les morceaux de l'écran sont rangés dans **Composants.kt**. Chacun est une petite fonction `@Composable` réutilisable.

Une ligne de la liste

Composants.kt

```
@Composable
fun LigneMouture(
    mouture: Mouture,
    selectionnee: Boolean,
    onClick: () -> Unit,
) {
    Card(
        onClick = onClick,
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = if (selectionnee) {
                MaterialTheme.colorScheme.primaryContainer
            } else {
                MaterialTheme.colorScheme.surfaceVariant
            },
        ),
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Text(
                text = mouture.nom,
                style = MaterialTheme.typography.titleMedium,
                modifier = Modifier.weight(1f),
            )
            Text(
                text = "SM ${mouture.sansMeule.vitesseBle}/${mouture.sansMeule.vitesseDoseur}" +
                    " · AM ${mouture.avecMeule.vitesseBle}/${mouture.avecMeule.vitesseDoseur}",
                style = MaterialTheme.typography.bodySmall,
            )
        }
    }
}
```

- `Card(onClick = …)` : une carte qu'on peut toucher (avec l'effet d'onde).
- `onClick: () -> Unit` : un paramètre de type **fonction**. La ligne ne sait pas ce qui se passe au clic ; elle prévient celui qui l'a créée.
- La couleur de fond change avec `selectionnee`.

La carte de détail

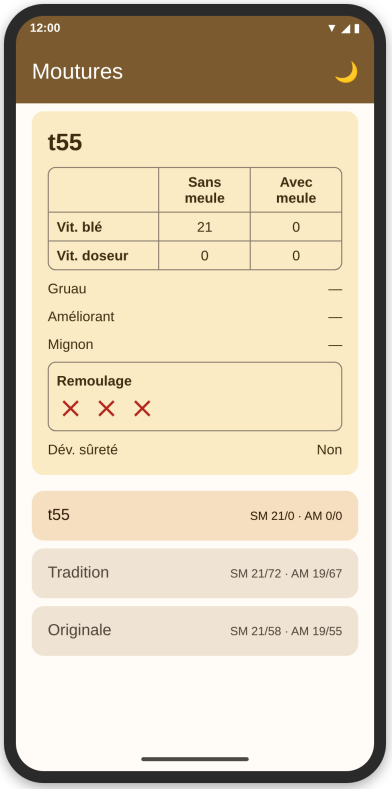
Composants.kt

```
@Composable
fun DetailMouture(mouture: Mouture) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = 8.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.secondaryContainer,
        ),
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp),
        ) {
            Text(
                text = mouture.nom,
                style = MaterialTheme.typography.headlineSmall,
                fontWeight = FontWeight.Bold,
            )

            TableauDosage(sansMeule = mouture.sansMeule, avecMeule = mouture.avecMeule)

            val info = mouture.info
            LigneInfo("Gruau", info.gruau ?: VIDE)
            LigneInfo("Améliorant", info.ameliorant ?: VIDE)
            LigneInfo("Mignon", info.mignon ?: VIDE)
            IconesRemoulage(titre = "Remoulage", valeurs = info.remoulage)
            LigneInfo("Dév. sûreté", if (info.devSurete) "Oui" else "Non")
        }
    }
}
```

`info.gruau ?: VIDE` : l'opérateur **Elvis** `?:` remplace une valeur `null` par `VIDE` (un tiret long « — »).



Avec la t55, gruau, améliorant et mignon valent `null` : la carte affiche « — ». Aucune case de remoulage n'est cochée : trois croix rouges.

Le tiret est rangé dans une constante :
`const val VIDE = "—"`
Si vous voulez le changer, une seule ligne à modifier.

Figure 7 — La carte de détail d'une mouture sans gruau ni améliorant.

Le tableau des vitesses

`weight(1.2f)` `weight(1f)` `weight(1f)`

	Sans meule	Avec meule
Vit. blé	21	19
Vit. doseur	72	67

`CaseTableau` `VerticalDivider`

1 Row = 1 ligne du tableau
Column = les 3 lignes empilées
même weight → colonnes alignées
traits rouges : VerticalDivider, visibles grâce à IntrinsicSize.Min

Figure 8 — Anatomie du tableau.

Composants.kt

```
@Composable
fun TableauDosage(sansMeule: Dosage, avecMeule: Dosage) {
    val trait = MaterialTheme.colorScheme.outline
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .border(1.dp, trait, RoundedCornerShape(8.dp)),
    ) {
        LigneTableau("", "Sans meule", "Avec meule", entete = true)
        HorizontalDivider(thickness = 1.dp, color = trait)
        LigneTableau(
            "Vit. blé",
            sansMeule.vitesseBle.toString(),
            avecMeule.vitesseBle.toString(),
        )
        HorizontalDivider(thickness = 1.dp, color = trait)
        LigneTableau(
            "Vit. doseur",
            sansMeule.vitesseDoseur.toString(),
            avecMeule.vitesseDoseur.toString(),
        )
    }
}
```

Composants.kt

```
@Composable
private fun LigneTableau(
    libelle: String,
    sansMeule: String,
    avecMeule: String,
    entete: Boolean = false,
) {
    val trait = MaterialTheme.colorScheme.outline
    // IntrinsicSize.Min donne une hauteur à la Row : sans ça, les traits verticaux sont invisibles.
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(IntrinsicSize.Min),
    ) {
        CaseTableau(libelle, Modifier.weight(1.2f), gras = true, alignement = TextAlign.Start)
        VerticalDivider(thickness = 1.dp, color = trait)
        CaseTableau(sansMeule, Modifier.weight(1f), gras = entete)
        VerticalDivider(thickness = 1.dp, color = trait)
        CaseTableau(avecMeule, Modifier.weight(1f), gras = entete)
    }
}
```

Piège fréquent
Sans `.height(IntrinsicSize.Min)`, les `VerticalDivider` sont invisibles : un trait vertical prend la hauteur de son parent, et la `Row` ne la connaît pas à l'avance.

Les icônes du remoulage

Composants.kt

```
@Composable
fun IconesRemoulage(
    titre: String,
    valeurs: List<Boolean>,
    modifier: Modifier = Modifier,
) {
    Column(
        modifier = modifier
            .fillMaxWidth()
            .border(1.dp, MaterialTheme.colorScheme.outline, RoundedCornerShape(8.dp))
            .padding(8.dp),
        verticalArrangement = Arrangement.spacedBy(4.dp),
    ) {
        Text(
            text = titre,
            style = MaterialTheme.typography.titleSmall,
            fontWeight = FontWeight.SemiBold,
        )
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            // Toujours 3 icônes : s'il manque une valeur, on la considère « non ».
            List(3) { i -> valeurs.getOrElse(i) { false } }.forEachIndexed { i, actif ->
                Icon(
                    imageVector = if (actif) Icons.Filled.CheckCircle else Icons.Filled.Close,
                    contentDescription = "Remoulage ${i + 1} : ${if (actif) "oui" else "non"}",
                    tint = if (actif) {
                        MaterialTheme.colorScheme.tertiary
                    } else {
                        MaterialTheme.colorScheme.error
                    },
                    modifier = Modifier.size(28.dp),
                )
            }
        }
    }
}
```

- `List(3) { … getOrElse(i) { false } }` : toujours 3 icônes, même s'il manque une valeur.
- `contentDescription` : texte lu à voix haute par TalkBack pour les personnes malvoyantes.
- Les couleurs viennent du thème (`tertiary` et `error`) : elles s'adaptent au mode sombre.

Piège fréquent
Importez `androidx.compose.material.icons.Icons` (sans « 3 »), et non `androidx.compose.material3.internal.Icons` que l'éditeur propose parfois : il est interne. La bibliothèque `material-icons-core` est déjà dans le projet.

9. Le thème clair et le thème sombre

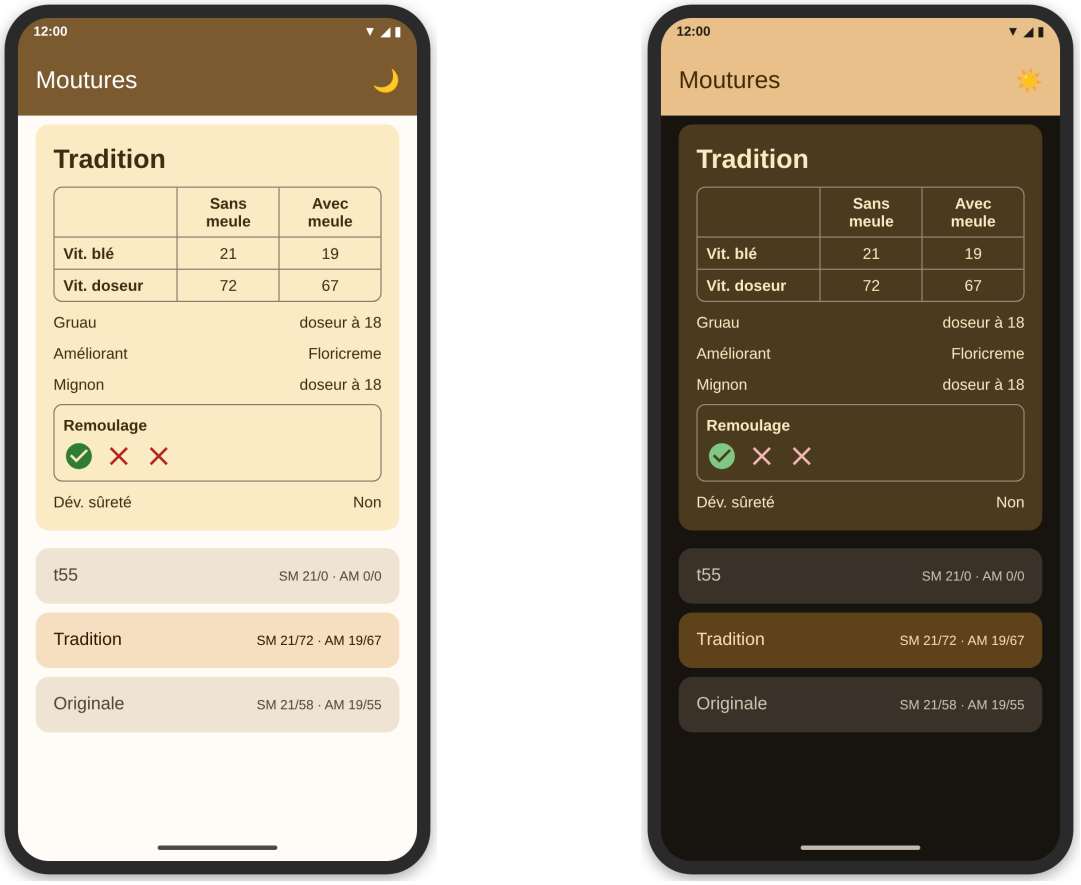


Figure 9 — Le même écran, le même code : seules les couleurs changent.

Material 3 range les couleurs dans un **ColorScheme**. Chaque couleur de fond a sa couleur de texte, du même nom précédé de **on**. On définit un jeu de couleurs pour le clair, un autre pour le sombre :

		Thème clair	Thème sombre
`primary`	`onPrimary`	Texte #7A5A2E	Texte #E9C08A
`primaryContainer`	`onPrimaryContainer`	Texte #F6DFC0	Texte #5E4219
`secondaryContainer`	`onSecondaryContainer`	Texte #FBEBC4	Texte #4A3B1E
`surfaceVariant`	`onSurfaceVariant`	Texte #EFE3D3	Texte #3A3128
`background`	`onBackground`	Texte #FFFBF6	Texte #17130E
`tertiary`	`(coche « oui »)`	Texte #2E7D32	Texte #81C784
`error`	`(croix « non »)`	Texte #B3261E	Texte #F2B8B5

Figure 10 — Les rôles de couleur utilisés par l'application, en clair et en sombre.

Étape 1 — Les couleurs

Color.kt

```
package com.example.moutures2.ui.theme

import androidx.compose.ui.graphics.Color

// ---------- Thème clair ----------
val ClairPrimary = Color(0xFF7A5A2E)            // brun farine
val ClairOnPrimary = Color(0xFFFFFFFF)
val ClairPrimaryContainer = Color(0xFFF6DFC0)
val ClairOnPrimaryContainer = Color(0xFF2B1700)
val ClairSecondaryContainer = Color(0xFFFBEBC4) // blé clair
val ClairOnSecondaryContainer = Color(0xFF3E2C10)
val ClairTertiary = Color(0xFF2E7D32)           // vert : « oui »
val ClairBackground = Color(0xFFFFFBF6)
val ClairOnBackground = Color(0xFF1F1B16)
val ClairSurfaceVariant = Color(0xFFEFE3D3)
val ClairOnSurfaceVariant = Color(0xFF4F4539)
val ClairOutline = Color(0xFF817567)
val ClairError = Color(0xFFB3261E)

// ---------- Thème sombre ----------
val SombrePrimary = Color(0xFFE9C08A)
val SombreOnPrimary = Color(0xFF442B05)
val SombrePrimaryContainer = Color(0xFF5E4219)
val SombreOnPrimaryContainer = Color(0xFFF6DFC0)
val SombreSecondaryContainer = Color(0xFF4A3B1E)
val SombreOnSecondaryContainer = Color(0xFFFBEBC4)
val SombreTertiary = Color(0xFF81C784)
val SombreBackground = Color(0xFF17130E)
val SombreOnBackground = Color(0xFFEBE1D6)
val SombreSurfaceVariant = Color(0xFF3A3128)
val SombreOnSurfaceVariant = Color(0xFFD3C4B2)
val SombreOutline = Color(0xFF9C8F80)
val SombreError = Color(0xFFF2B8B5)
```

Étape 2 — Le thème

Theme.kt

```
package com.example.moutures2.ui.theme

import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable

private val CouleursClaires = lightColorScheme(
    primary = ClairPrimary,
    onPrimary = ClairOnPrimary,
    primaryContainer = ClairPrimaryContainer,
    onPrimaryContainer = ClairOnPrimaryContainer,
    secondaryContainer = ClairSecondaryContainer,
    onSecondaryContainer = ClairOnSecondaryContainer,
    tertiary = ClairTertiary,
    background = ClairBackground,
    onBackground = ClairOnBackground,
    surface = ClairBackground,
    onSurface = ClairOnBackground,
    surfaceVariant = ClairSurfaceVariant,
    onSurfaceVariant = ClairOnSurfaceVariant,
    outline = ClairOutline,
    error = ClairError,
)

private val CouleursSombres = darkColorScheme(
    primary = SombrePrimary,
    onPrimary = SombreOnPrimary,
    primaryContainer = SombrePrimaryContainer,
    onPrimaryContainer = SombreOnPrimaryContainer,
    secondaryContainer = SombreSecondaryContainer,
    onSecondaryContainer = SombreOnSecondaryContainer,
    tertiary = SombreTertiary,
    background = SombreBackground,
    onBackground = SombreOnBackground,
    surface = SombreBackground,
    onSurface = SombreOnBackground,
    surfaceVariant = SombreSurfaceVariant,
    onSurfaceVariant = SombreOnSurfaceVariant,
    outline = SombreOutline,
    error = SombreError,
)

/**
 * Le thème de l'application.
 * darkTheme = true  → couleurs sombres
 * darkTheme = false → couleurs claires
 * Par défaut, on suit le réglage du téléphone.
 */
@Composable
fun MouturesTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit,
) {
    MaterialTheme(
        colorScheme = if (darkTheme) CouleursSombres else CouleursClaires,
        content = content,
    )
}
```

- `lightColorScheme(…)` et `darkColorScheme(…)` créent les deux jeux. Les rôles non précisés gardent une valeur par défaut de Material 3.

- `darkTheme: Boolean = isSystemInDarkTheme()` : par défaut, on suit le réglage du téléphone.
- Tout ce qui est à l'intérieur de `MouturesTheme { … }` utilise ces couleurs via `MaterialTheme.colorScheme`.

> **Bon à savoir**
> On n'utilise pas les **couleurs dynamiques** d'Android 12+ (tirées du fond d'écran) : elles remplaceraient les couleurs brun et blé de l'application.

Étape 3 — Le bouton clair / sombre

MainActivity.kt

```
package com.example.moutures2

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.SystemBarStyle
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import com.example.moutures2.ui.MouturesScreen
import com.example.moutures2.ui.theme.MouturesTheme

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        setContent {
            // Au démarrage, on prend le réglage du téléphone (clair ou sombre).
            val systemeSombre = isSystemInDarkTheme()
            // rememberSaveable : le choix est conservé si on tourne l'écran.
            var sombre by rememberSaveable { mutableStateOf(systemeSombre) }

            // Couleur des icônes système (heure, batterie, barre de navigation).
            // En thème clair, la barre du haut est brun foncé : icônes blanches en haut.
            // En thème sombre, elle est beige clair : icônes foncées en haut.
            DisposableEffect(sombre) {
                val transparent = android.graphics.Color.TRANSPARENT
                val iconesClaires = SystemBarStyle.dark(transparent)
                val iconesFoncees = SystemBarStyle.light(transparent, transparent)
                enableEdgeToEdge(
                    statusBarStyle = if (sombre) iconesFoncees else iconesClaires,
                    navigationBarStyle = if (sombre) iconesClaires else iconesFoncees,
                )
                onDispose { }
            }

            MouturesTheme(darkTheme = sombre) {
                MouturesScreen(
                    sombre = sombre,
                    onSombreChange = { sombre = it },
                )
            }
        }
    }
}
```

Ligne	Explication
`isSystemInDarkTheme()`	Le réglage du téléphone au démarrage.
`rememberSaveable { mutableStateOf(…) }`	Une variable d'état qui **survit à la rotation**. Quand elle change, l'écran se redessine.
`onSombreChange = { sombre = it }`	Le bouton de la barre du haut appelle cette fonction ; `it` est la nouvelle valeur.
`DisposableEffect(sombre)`	Relancé à chaque changement de thème : règle la couleur des icônes système (heure, batterie) pour qu'elles restent lisibles.
`MouturesTheme(darkTheme = sombre)`	Applique le bon jeu de couleurs à tout l'écran.

Étape 4 — Le thème de démarrage

Pendant une fraction de seconde, avant que Compose ne dessine, Android affiche la fenêtre avec le thème XML. Deux fichiers portant le même nom de style évitent un flash blanc en mode sombre : Android choisit tout seul `values-night` quand le téléphone est en sombre.

res/values/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Thème de la fenêtre au démarrage (avant que Compose ne dessine) : clair. -->
    <style name="Theme.Moutures" parent="android:Theme.Material.Light.NoActionBar">
        <item name="android:windowBackground">@android:color/white</item>
    </style>
</resources>
```

res/values-night/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Même thème quand le téléphone est en mode sombre. -->
    <style name="Theme.Moutures" parent="android:Theme.Material.NoActionBar">
        <item name="android:windowBackground">@android:color/black</item>
    </style>
</resources>
```

10. Les aperçus @Preview

Dans Android Studio, une fonction marquée `@Preview` s'affiche dans le panneau **Split** ou **Design**, **sans lancer l'application**. Le projet en contient deux : clair et sombre.

MouturesScreen.kt

```
@Preview(name = "Thème clair", showBackground = true)
@Composable
private fun ApercuClair() {
    MouturesTheme(darkTheme = false) {
        MouturesContent(
            etat = MouturesUiState(
                moutures = MouturesData.moutures,
                selection = MouturesData.moutures[1],
            ),
            sombre = false,
            onSombreChange = {},
            onMoutureClick = {},
        )
    }
}
```

- On appelle `MouturesContent` avec des données fixes et des fonctions vides `{}` : pas besoin de ViewModel.
- `private` : l'aperçu n'est utile que dans ce fichier.
- Après une modification, cliquez sur **Build & Refresh** si l'aperçu ne se met pas à jour.

11. Les bibliothèques

app/build.gradle.kts

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
}

android {
    namespace = "com.example.moutures2"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.moutures2"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }

    kotlinOptions {
        jvmTarget = "17"
    }

    buildFeatures {
        compose = true
    }
}

dependencies {
    // Le BOM fixe des versions compatibles pour toutes les bibliothèques Compose.
    implementation(platform("androidx.compose:compose-bom:2024.12.01"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.ui:ui-graphics")
    implementation("androidx.compose.ui:ui-tooling-preview")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-core")

    implementation("androidx.core:core-ktx:1.15.0")
    implementation("androidx.activity:activity-compose:1.9.3")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")

    debugImplementation("androidx.compose.ui:ui-tooling")
}
```

Bibliothèque	Ce qu'elle apporte
`compose-bom`	Fixe des versions compatibles pour toutes les bibliothèques Compose.
`material3`	Card, Scaffold, TopAppBar, Text, les couleurs Material 3…
`material-icons-core`	Les icônes `CheckCircle` et `Close`.

Bibliothèque	Ce qu'elle apporte
`activity-compose`	`setContent`, `enableEdgeToEdge`.
`lifecycle-viewmodel-compose`	`ViewModel` et `viewModel()`.
`lifecycle-runtime-compose`	`collectAsStateWithLifecycle()`.
`ui-tooling`	Les aperçus `@Preview` (version debug uniquement).

> **Bon à savoir**
> Après chaque modification de ce fichier, cliquez sur **Sync Now**.

12. Erreurs fréquentes et solutions

Message ou symptôme	Cause	Solution
No value passed for parameter '...'	Un champ oublié dans un `Mouture(…)`.	Ajouter le champ manquant.
Type mismatch: inferred type is String but Int was expected	Un nombre écrit entre guillemets : `vitesseBle = "21"`.	Retirer les guillemets : `vitesseBle = 21`.
Null can not be a value of a non-null type String	`null` dans un champ sans `?`.	Ajouter `?` au type, ou donner un texte.
Expression 'weight' ... cannot be invoked as a function	`weight` hors d'une Row/Column.	Entourer d'une `Row`/`Column`, ou utiliser `fillMaxWidth()`.
@Composable invocations can only happen...	Composable appelé directement dans une `LazyColumn`.	L'entourer de `item { … }`.
Cannot access 'Icons': it is internal	Mauvais import `material3.internal`.	Importer `androidx.compose.material.icons.Icons`.
Unresolved reference	Import ou bibliothèque manquant.	Alt+Entrée sur le mot rouge, ou Sync Gradle.
Traits verticaux invisibles	Row sans hauteur connue.	`.height(IntrinsicSize.Min)`.
Le thème ne change pas	`MouturesTheme` sans `darkTheme = sombre`.	Passer la variable au thème.
Heure illisible en haut	Icônes système de la même couleur que la barre.	Le `DisposableEffect` de MainActivity règle ce point.

13. Exercices

Faites-les dans l'ordre. Les indices sont en bas de page.

Exercice 1 — Ajouter une mouture
Ajoutez « Seigle » dans `MouturesData` avec vos valeurs. Elle doit apparaître en fin de liste.

Exercice 2 — Provoquer une erreur
Retirez `devSurete = false,` d'une mouture. Lisez le message, puis réparez.

Exercice 3 — Changer une couleur
Changez `ClairPrimary` en vert foncé `0xFF2E5E2E`. Observez la barre du haut dans l'aperçu clair.

Exercice 4 — Compteur
Affichez « 3 moutures » (le nombre réel) en tout premier dans la liste.

Exercice 5 — Bascule
Un second toucher sur la mouture sélectionnée doit refermer la carte.

Exercice 6 — Ligne Total
Ajoutez au tableau une ligne « Total » : vitesse blé + vitesse doseur.

Exercice 7 — Thème au choix (défi)
Ajoutez un troisième choix « Système » qui suit le réglage du téléphone.

Indices

1. Copiez le bloc `Mouture(…),` de la t55 et collez-le à la fin de `listOf`.
2. Le message dit exactement quel paramètre manque.
3. Les deux aperçus (`@Preview`) se mettent à jour après un Build & Refresh.
4. `item { Text("${etat.moutures.size} moutures") }` avant la carte de détail.
5. Dans le ViewModel : `selection = if (etat.selection?.nom == mouture.nom) null else mouture`.
6. `LigneTableau("Total", (sansMeule.vitesseBle + sansMeule.vitesseDoseur).toString(), …)`.
7. Remplacez le `Boolean` par une `enum class ChoixTheme { CLAIR, SOMBRE, SYSTEME }`.

14. Glossaire

Terme	Définition
Activity	Un écran Android. Ici une seule : `MainActivity`.
BOM	Liste de versions compatibles pour les bibliothèques Compose.
ColorScheme	L'ensemble des couleurs Material 3 (primary, onPrimary…).
Composable	Fonction `@Composable` qui décrit un morceau d'interface.
data class	Classe Kotlin faite pour contenir des données, avec `copy()`.
dp / sp	Unités de taille ; sp suit en plus la taille de police de l'utilisateur.
État (state)	Ce que l'écran affiche à un instant donné.
LazyColumn	Liste qui défile et ne dessine que les éléments visibles.
Modifier	Réglages d'un composable : taille, marges, fond, clic… L'ordre compte.
null	Absence de valeur. Seul un type avec `?` peut la contenir.
object	Un objet unique, disponible partout (ici `MouturesData`).
Preview	Aperçu d'un composable dans Android Studio, sans lancer l'appli.
Recomposition	Compose rappelle les composables dont l'état a changé.
StateFlow	Boîte qui contient une valeur et prévient de ses changements.
Thème sombre	Jeu de couleurs foncées, plus reposant la nuit et économe sur écran OLED.
ViewModel	Garde l'état de l'écran ; survit à la rotation.

Les illustrations d'écran de ce cours sont des maquettes qui reprennent exactement les couleurs et les dimensions du code. Le rendu réel peut varier légèrement selon le téléphone (police, barre système).